

Experiment No: 2 Exporatory Data Analysis(import and export files)

Aim:

To import and export different types of files using python.

Algorithm:

1. **Sample** – Select a representative subset of data from the dataset.
2. **Explore** – Analyze the data to understand patterns, relationships, and detect missing values or outliers.
3. **Modify** – Clean, transform, and prepare the data by creating or selecting useful features.
4. **Model** – Apply machine learning algorithms to build predictive models.
5. **Assess** – Evaluate the model's performance using appropriate metrics and choose the best model.

Code:

```
" -----
# SEMMA FRAMEWORK
# =====

# Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# =====
# S - SAMPLE
# =====

print("===== SAMPLE =====")

# Display first few records
print(df.head())

# Dataset shape
print("\nDataset Shape:")
print(df.shape)

# Dataset columns
print("\nColumns:")
print(df.columns)
```

```

# E - EXPLORE
# =====

print("\n===== EXPLORE =====")

# Dataset Information
print(df.info())

# Statistical Summary
print(df.describe())

# Missing Values
print("\nMissing Values")
print(df.isnull().sum())

# Correlation Matrix
plt.figure(figsize=(8,6))
corr = df.corr(numeric_only=True)

plt.imshow(corr, cmap='coolwarm')
plt.colorbar()

plt.xticks(range(len(corr.columns)), corr.columns, rotation=90)
plt.yticks(range(len(corr.columns)), corr.columns)

plt.title("Correlation Matrix")
plt.tight_layout()
plt.show()

# Likes Distribution
plt.figure(figsize=(6,4))
plt.hist(df["Likes"], bins=20)

# =====
# M - MODIFY
# =====

print("\n===== MODIFY =====")

# Select Features and Target
X = df.drop("Likes", axis=1)

y = df["Likes"]

print("Features Selected:")
print(X.columns)

# =====
# M - MODEL
# =====

print("\n===== MODEL =====")

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

# Build Model
model = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)

```

```

# Train Model
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

print("Model Training Completed Successfully.")

# =====
# A - ASSESS
# =====

print("\n===== ASSESS =====")

# Evaluation Metrics

mae = mean_absolute_error(y_test, y_pred)

rmse = np.sqrt(mean_squared_error(y_test, y_pred))

r2 = r2_score(y_test, y_pred)

print("Mean Absolute Error :", mae)

print("Root Mean Squared Error :", rmse)

print("R2 Score :", r2)

# Feature Importance
# =====

importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": model.feature_importances_
})

importance = importance.sort_values(
    by="Importance",
    ascending=False
)

print("\nFeature Importance")

print(importance)

# =====
# Feature Importance Graph
# =====

plt.figure(figsize=(8,5))

plt.bar(
    importance["Feature"],
    importance["Importance"]
)

plt.xlabel("Features")
plt.ylabel("Importance")
plt.title("Feature Importance")

plt.xticks(rotation=45)
plt.tight_layout()

plt.show()

print("\n===== SEMMA COMPLETED =====")

```

Output:

===== SAMPLE =====

	Post_ID	Platform	Likes	Comments	Shares	Views	Sentiment
0	1	1	304.0	384.0	71	17049	2
1	2	0	939.0	351.0	190	36741	2
2	3	4	250000.0	21.0	8	7140	2
3	4	0	4239.0	313.0	7	37781	2
4	5	4	3536.0	117.0	115	39618	1

Dataset Shape:

(46, 7)

Columns:

```
Index(['Post_ID', 'Platform', 'Likes', 'Comments', 'Shares', 'Views',  
      'Sentiment'],  
      dtype='object')
```

===== EXPLORE =====

```
<class 'pandas.core.frame.DataFrame'>
```

```
Index: 46 entries, 0 to 49
```

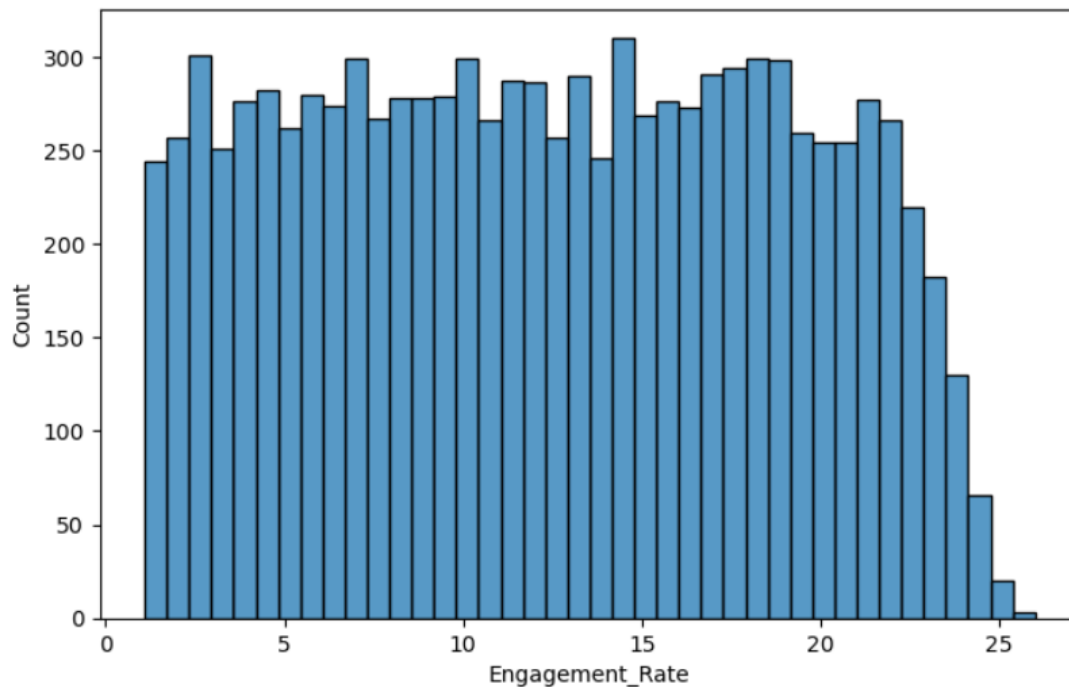
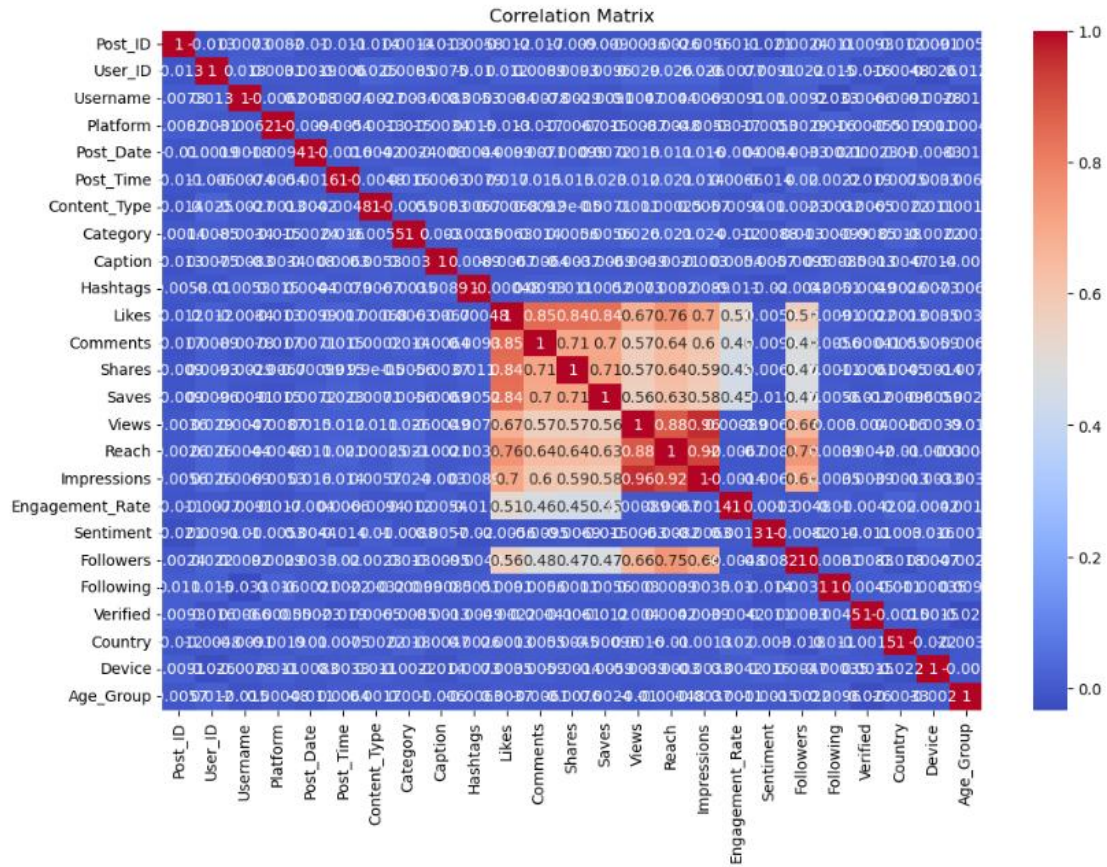
```
Data columns (total 7 columns):
```

#	Column	Non-Null Count	Dtype
0	Post_ID	46 non-null	int64
1	Platform	46 non-null	int64
2	Likes	46 non-null	float64
3	Comments	46 non-null	float64
4	Shares	46 non-null	int64
5	Views	46 non-null	int64
6	Sentiment	46 non-null	int64

```
dtypes: float64(2), int64(5)
```

```
memory usage: 2.9 KB
```

```
None
```



CRISP-DM

Algorithm

1. Business Understanding
2. Data Understanding
3. Data Preparation (already completed)
4. Modeling
5. Evaluation
6. Deployment

Code

```
print("Business Goal: Predict Likes")

X=df.drop("Likes",axis=1)
y=df["Likes"]

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor

X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)

model=RandomForestRegressor(random_state=42)
model.fit(X_train,y_train)

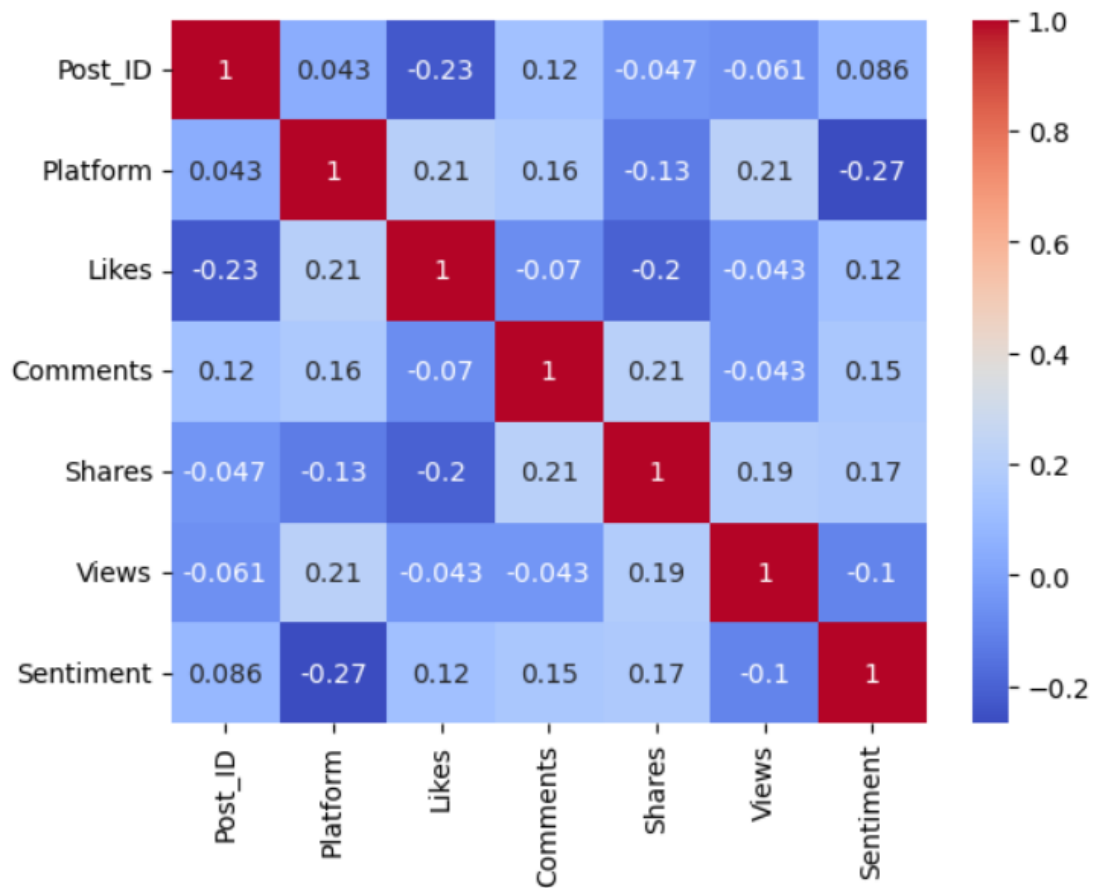
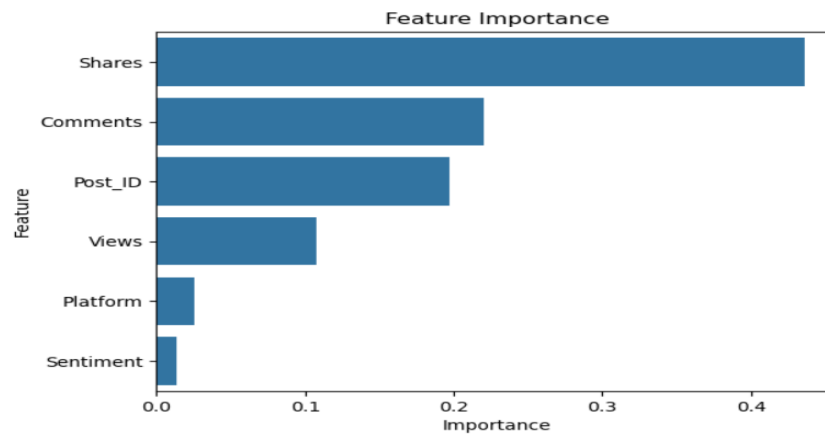
y_pred=model.predict(X_test)

from sklearn.metrics import mean_absolute_error,mean_squared_error,r2_score
import numpy as np
print("MAE:",mean_absolute_error(y_test,y_pred))
print("RMSE:",np.sqrt(mean_squared_error(y_test,y_pred)))
print("R2:",r2_score(y_test,y_pred))
```

Business Goal: Predict Likes
MAE: 19564.716999999997
RMSE: 40103.873798758774
R2: -732.903577548309

```
sns.barplot(
    data=importance,
    x="Importance",
    y="Feature"
)

plt.title("Feature Importance")
plt.show()
```



KDD

Algorithm

1. Data Selection
2. Data Transformation
3. Data Mining
4. Pattern Evaluation
5. Knowledge Presentation

Code

```
[52]: # Data Selection
print(df.head())

# Data Transformation
X=df.drop("Likes",axis=1)
y=df["Likes"]

# Data Mining
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor

X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)

model=RandomForestRegressor(random_state=42)
model.fit(X_train,y_train)

y_pred=model.predict(X_test)

# Pattern Evaluation
from sklearn.metrics import mean_absolute_error,mean_squared_error,r2_score
import numpy as np
print("MAE:",mean_absolute_error(y_test,y_pred))
print("RMSE:",np.sqrt(mean_squared_error(y_test,y_pred)))
print("R2:",r2_score(y_test,y_pred))

# Knowledge Presentation
import pandas as pd
importance=pd.DataFrame({"Feature":X.columns,"Importance":model.feature_importances_})
print(importance.sort_values(by="Importance",ascending=False))
```

Output

	Post_ID	Platform	Likes	Comments	Shares	Views	Sentiment
0	1	1	304.0	384.0	71	17049	2
1	2	0	939.0	351.0	190	36741	2
2	3	4	250000.0	21.0	8	7140	2
3	4	0	4239.0	313.0	7	37781	2
4	5	4	3536.0	117.0	115	39618	1

MAE: 19564.716999999997

RMSE: 40103.873798758774

R2: -732.903577548309

	Feature	Importance
3	Shares	0.435930
2	Comments	0.220547
0	Post_ID	0.196712
4	Views	0.107214
1	Platform	0.025707
5	Sentiment	0.013890

